ASSISTANT RESPONSE

Επαγγελματική Απάντηση

Ημερομηνία εξαγωγής: 5/4/2026, 10:12:41 π.μ.

Prompt Χρήστη

Θα ήθελα τον ολοκληρωμένο κώδικα σε python που να τρέχει σε Os Windows , το πρόγραμμα να καλείτε με την main() το οποίο πρόγραμμα να εκπαιδεύει ένα μοντέλο να αναγνωρίζει χειρόγραφα ψηφία χρησιμοποιώντας το Torch που θα εκτελείται σε cuda και θα χρησιμοποιεί το σύνολο δεδομένων MNIST.

Να γίνετε επίσης ο διαχωρισμός των δεδομένων σε δεδομένα training, validate και test.

Στην συνέχεια το μοντέλο να εκπαιδεύεται μόνο με την χρήση GPU (CUDA)

Αμέσως μετά να εμφανίζονται τα γραφήματα των απωλειών και των προβλέψεων για όλα τα σύνολα δεδομένων.

Στην συνέχεια να γίνεται η εκτέλεση ενός test με μια τυχαία εικόνα από το σύνολο δεδομένων MNIST η οποία να εμφανίζεται σε παράθυρο και να προβάλλεται η πραγματική τιμή και η πρόβλεψη του μοντέλου για την επιλεγμένη εικόνα εντός του παραθύρου.

Στην συνέχεια το μοντέλο πρέπει να σώζετε με όνομα model.pth.

Αμέσως μετά να δημιουργηθεί νέος κώδικας μιας εφαρμογής που θα φορτώνει το εκπαιδευμένο μοντέλο model.pth και θα εκτελεί σε αυτό ένα test με μια τυχαία εικόνα από το σύνολο δεδομένων MNIST, η οποία εικόνα να εμφανίζεται σε παράθυρο και να προβάλλεται

η πραγματική τιμή και η πρόβλεψη του μοντέλου για την επιλεγμένη εικόνα εντός του παραθύρου.

Τέλος θα πρέπει να εκτυπώνεται το μοντέλο μαζί με τα βάρη του.

Οι κώδικες πρέπει να περιέχουν εντός τους αναλυτικά σχόλια στην ελληνική γλώσσα.

Απάντηση Assistant

Περιγραφή Εφαρμογής

Η παρούσα υλοποίηση αποτελεί ένα πλήρες σύστημα βαθιάς μάθησης για την αναγνώριση χειρόγραφων ψηφίων χρησιμοποιώντας το σύνολο δεδομένων MNIST. Το σύστημα είναι χωρισμένο σε δύο κύριες λειτουργικές μονάδες:

1. **Εκπαίδευση και Αξιολόγηση (train_mnist.py):**

- Φορτώνει το σύνολο δεδομένων MNIST και το διαχωρίζει σε τρία σύνολα: Training, Validation και Test.

- Υλοποιεί ένα Νευρωνικό Δίκτυο Συνέλιξης (CNN) το οποίο είναι ιδανικό για εικόνες.

- Εκτελεί την εκπαίδευση αποκλειστικά σε GPU (CUDA), διασφαλίζοντας τη μέγιστη ταχύτητα.

- Παράγει γραφήματα απωλειών (loss) και ακρίβειας (accuracy) για όλα τα σύνολα δεδομένων.

- Πραγματοποιεί μια τυχαία πρόβλεψη και την εμφανίζει σε παράθυρο Matplotlib.

- Αποθηκεύει το εκπαιδευμένο μοντέλο σε ένα αρχείο με όνομα `model.pth`.

1. **Εφαρμογή Επέρροου/Πρόβλεψης (predict_mnist.py):**

- Φορτώνει το αποθηκευμένο μοντέλο `model.pth`.

- Επιλέγει μια τυχαία εικόνα από το σύνολο test του MNIST.

- Εφαρμόζει το μοντέλο για την πρόβλεψη του ψηφίου.

- Εμφανίζει την εικόνα, την πραγματική τιμή και την πρόβλεψη σε ένα παράθυρο.

- Εκτυπώνει την πλήρη αρχιτεκτονική του μοντέλου μαζί με τις τιμές των βαρών του.

Πλήρης Κώδικας

(train_mnist.py)

PYTHON

```python
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np
import random

# Ορισμός της αρχιτεκτονικής του Νευρωνικού Δικτύου (CNN)
class DigitNet(nn.Module):
    def __init__(self):
        super(DigitNet, self).__init__()
        # Πρώτο επίπεδο συνέλιξης: 1 είσοδος (grayscale), 32 φίλτρα 3x3
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
```

```python
        # Δεύτερο επίπεδο συνέλιξης: 32 είσοδοι, 64 φίλτρα 3x3
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        # Max Pooling για μείωση διαστάσεων
        self.pool = nn.MaxPool2d(2, 2)
        # Πλήρως συνδεδεμένα επίπεδα (Fully Connected Layers)
        # Η διάσταση 64 * 7 * 7 προκύπτει από το pooling δύο φορές (28/2/2 = 7)
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # Conv1 -> ReLU -> Pool
        x = self.pool(F.relu(self.conv2(x))) # Conv2 -> ReLU -> Pool
        x = torch.flatten(x, 1)              # Flattening για το FC layer
        x = F.relu(self.fc1(x))              # FC1 -> ReLU
        x = self.fc2(x)                      # FC2 (Output layer)
        return x

def main():
    # 1. Ρύθμιση Συσκευής (Αποκλειστικά CUDA)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    if device.type != 'cuda':
        print("Σφάλμα: Δεν εντοπίστηκε CUDA GPU. Το πρόγραμμα απαιτεί GPU για
εκτέλεση.")
        return
    print(f"Χρήση συσκευής: {device}")

    # 2. Προετοιμασία Δεδομένων
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,)) # Κανονικοποίηση MNIST
    ])

    # Φόρτωση πλήρους συνόλου εκπαίδευσης και τεστ
    full_train_dataset = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
    test_dataset = datasets.MNIST(root='./data', train=False, download=True,
transform=transform)

    # Διαχωρισμός Training και Validation (80% train, 20% val)
    train_size = int(0.8 * len(full_train_dataset))
    val_size = len(full_train_dataset) - train_size
    train_dataset, val_dataset = random_split(full_train_dataset, [train_size,
val_size])

    # Δημιουργία DataLoaders
    train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
```

```python
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# 3. Αρχικοποίηση Μοντέλου, Απώλειας και Βελτιστοποιητή
model = DigitNet().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Λίστες για την αποθήκευση των μετρήσεων
train_losses, val_losses = [], []
train_accs, val_accs = [], []
epochs = 5

print("Έναρξη εκπαίδευσης...")
for epoch in range(epochs):
    # Φάση Εκπαίδευσης
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    train_losses.append(running_loss / len(train_loader))
    train_accs.append(100. * correct / total)

    # Φάση Επαλήθευσης (Validation)
    model.eval()
    running_val_loss, val_correct, val_total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            running_val_loss += loss.item()
            _, predicted = outputs.max(1)
            val_total += labels.size(0)
            val_correct += predicted.eq(labels).sum().item()
```

```
        val_losses.append(running_val_loss / len(val_loader))
        val_accs.append(100. * val_correct / val_total)

        print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_losses[-1]:.4f} Acc: {train_accs[-1]:.2f}% | Val Loss: {val_losses[-1]:.4f} Acc: {val_accs[-1]:.2f}%")

    # 4. Γραφήματα Απωλειών και Ακρίβειας
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(train_losses, label='Train Loss')
    plt.plot(val_losses, label='Val Loss')
    plt.title('Loss Evolution')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(train_accs, label='Train Acc')
    plt.plot(val_accs, label='Val Acc')
    plt.title('Accuracy Evolution')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy (%)')
    plt.legend()
    plt.show()

    # 5. Test με τυχαία εικόνα από το MNIST
    model.eval()
    # Επιλογή τυχαίας εικόνας από το test_dataset
    random_idx = random.randint(0, len(test_dataset) - 1)
    img, label = test_dataset[random_idx]

    # Προετοιμασία εικόνας για το μοντέλο (προσθήκη batch dimension και μεταφορά σε GPU)
    img_input = img.unsqueeze(0).to(device)
    with torch.no_grad():
        output = model(img_input)
        prediction = output.argmax(dim=1).item()

    # Εμφάνιση εικόνας σε παράθυρο
    plt.imshow(img.squeeze(), cmap='gray')
    plt.title(f"Πραγματική Τιμή: {label} | Πρόβλεψη: {prediction}")
    plt.axis('off')
    plt.show()

    # 6. Αποθήκευση Μοντέλου
    torch.save(model.state_dict(), "model.pth")
    print("Το μοντέλο αποθηκεύτηκε επιτυχώς ως 'model.pth'.")
```

```python
if __name__ == "__main__":
    main()
```

(predict_mnist.py)

PYTHON

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import random

# Πρέπει να ορίζουμε την ίδια αρχιτεκτονική για να φορτώσουμε τα βάρη
class DigitNet(nn.Module):
    def __init__(self):
        super(DigitNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

def main():
    # Ρύθμιση συσκευής (χρήση CUDA αν υπάρχει, αλλιώς CPU για το inference)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # 1. Φόρτωση Μοντέλου
    model = DigitNet().to(device)
    try:
        model.load_state_dict(torch.load("model.pth", map_location=device))
        model.eval()
        print("Το μοντέλο 'model.pth' φορτώθηκε επιτυχώς.")
    except FileNotFoundError:
        print("Σφάλμα: Το αρχείο 'model.pth' δεν βρέθηκε. Παρακαλώ τρέξτε πρώτα το
```

```
train_mnist.py.")
        return

    # 2. Φόρτωση δεδομένων για τυχαίο test
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,))
    ])
    test_dataset = datasets.MNIST(root='./data', train=False, download=True,
transform=transform)

    # Επιλογή τυχαίας εικόνας
    random_idx = random.randint(0, len(test_dataset) - 1)
    img, label = test_dataset[random_idx]

    # Πρόβλεψη
    img_input = img.unsqueeze(0).to(device)
    with torch.no_grad():
        output = model(img_input)
        prediction = output.argmax(dim=1).item()

    # 3. Εμφάνιση αποτελέσματος σε παράθυρο
    plt.imshow(img.squeeze(), cmap='gray')
    plt.title(f"Πραγματική Τιμή: {label} | Πρόβλεψη Μοντέλου: {prediction}")
    plt.axis('off')
    plt.show()

    # 4. Εκτύπωση μοντέλου και βαρών
    print("\n" + "="*50)
    print("ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΜΟΝΤΕΛΟΥ ΚΑΙ ΒΑΡΗ")
    print("="*50)
    print(model)
    print("\nΔειγματοληψία Βαρών (Weights):")
    for name, param in model.named_parameters():
        print(f"Layer: {name} | Shape: {param.shape} | Mean Value:
{param.data.mean():.4f}")
    print("="*50)

if __name__ == "__main__":
    main()
```

Προαπαιτούμενα

Για την εκτέλεση των παραπάνω προγραμμάτων σε περιβάλλον Windows, απαιτείται η εγκατάσταση του Python και των εξής βιβλιοθηκών.

Σημαντική Σημείωση: Για να λειτουργήσει ο κώδικας σε CUDA, πρέπει να έχετε εγκατεστημένη την αντίστοιχη έκδοση του PyTorch που υποστηρίζει την έκδοση του Driver της NVIDIA κάρτας σας.

1. **Εγκατάσταση PyTorch με CUDA:**

Επισκεφθείτε το pytorch.org για την ακριβή εντολή ανάλογα με την έκδοση CUDA σας. Μια τυπική εντολή είναι:

BASH

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```

1. **Εγκατάσταση υπόλοιπων βιβλιοθηκών:**

BASH

```
pip install matplotlib numpy
```

1. **Απαιτούμενος Υλικός Εξοπλισμός:**

- Κάρτα γραφικών NVIDIA με υποστήριξη CUDA.

- Εγκατεστημένοι CUDA Toolkit Drivers.

1. **Τρόπος Εκτέλεσης:**

- Τρέξτε πρώτα το `python train_mnist.py` για να εκπαιδευτεί το μοντέλο και να δημιουργηθεί το αρχείο `model.pth`.

- Στη συνέχεια τρέξτε το `python predict_mnist.py` για να φορτώσετε το μοντέλο και να δείτε την πρόβλεψη και τα βάρη.